

ТУРМАЛИН КОД

Инженерный журнал
о коде, процессах
и бизнес-решениях

test-driven
development

{РАЗРАБОТКА ЧЕРЕЗ
ТЕСТИРОВАНИЕ}

```
def test_premium_user_gets_10_percent_off_order_total():
    order = order_with(
        items=[
            ("book", 500),
            ("book", 500),
            ("magazine Tourmaline Code", 1),

            ("TDD extreme programming", 10),
            ("Code without bugs", 10),
            ("Code sustainability", 1000),
        ],
        user=premium_user(),
    )

    user_name = order.get_username()
    user = UserCommand.get_by_username(user_name)
    assert user.type == "premium"

    total = order.total()
    assert total == 900
```

{В ЭТОМ ВЫПУСКЕ}

TDD В РАЗНЫХ
НАПРАВЛЕНИЯХ
РАЗРАБОТКИ

УСЛОВИЯ,
ПРИ КОТОРЫХ
TDD РАБОТАЕТ

№1

09.2026

Test-Driven Development (разработка через тестирование)

Привет!

Мы — Tourmaline Core.

7 лет мы делаем IT для очень разных компаний и организаций. Carl Zeiss и Челябинский зоопарк, медицинские стартапы и австралийские благотворители. Драйверы, ML-модели, веб-системы, прошивки, блокчейн...

За это время накопилось много историй: про то, как принимались решения, что из этого получалось, где мы ошибались и какие выводы помогли делать следующие проекты иначе. Так шпала за шпалой прокладывались рельсы, на которые мы ставим новые проекты.

Часть из наших историй становится статьями на Хабре. Часть — докладами на конференциях. Но какие-то кусочки пазла не помещаются в эти форматы: это истории, у которых много участников, много отсылок, и в которых одна тема цепляется за десяток других. Из таких историй и будет собираться этот журнал.

Первый выпуск про Test-Driven Development. Тему мы выбрали не случайно: несколько лет назад наш руководитель принял принципиальное решение — дальше работаем по TDD везде, где это возможно. Сейчас это основа нашего инженерного фокуса.

И на каждом проекте это решение мы объясняем как техническим специалистам со стороны заказчика, так и управленцам. Решили, что пора собрать наш ответ обеим сторонам в одном месте.

Мария Ядрышникова,
Директор по развитию



Что такое TDD?

В 2001 году журнал «Хакер» опубликовал легендарный материал «Если бы программисты строили дома». Бригада забыла фундамент и построила первый этаж, пришлось все ломать. Потом на пятом этаже стена ушла под 40 градусов. Ещё неправильно положили какой-то кирпич, а какой именно — непонятно, проще снести и переделывать. Крыша строилась отдельно и не подошла к дому. Лифт после оптимизации пробил крышу и улетел.

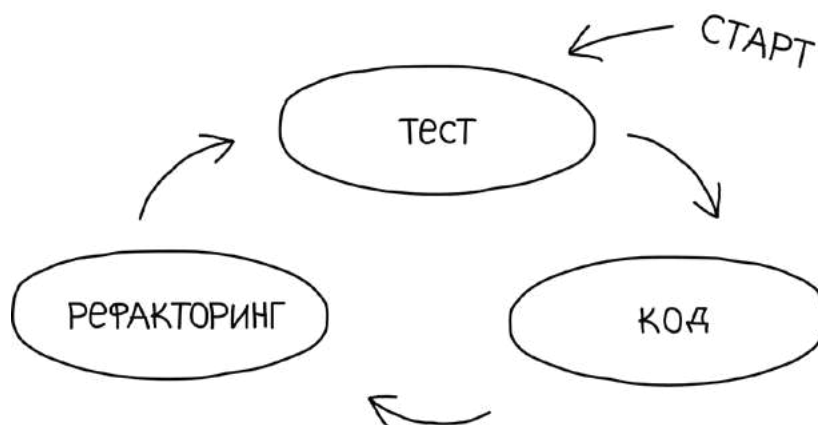
Конечно, перед стройкой настоящего дома всё проектируют до старта. Договариваются обо всём: сколько этажей, какая нагрузка на перекрытия, где несущие стены, какую делать отделку и из чего. Техническое задание — это формулировка того, каким должен получиться дом. В разработке ПО часто делают наоборот. Программист пишет код, а потом пишет автоматизированные тесты к тому, что уже получилось. И то не всегда. Эти тесты подтверждают принятые в процессе разработки решения, а не помогают их проектировать. Поэтому часто ИТ-системы — это дом из байки «Хакера».

TDD (Test-Driven Development) — это разработка, в которой сначала договариваются, потом делают.

Программист записывает, как новая функция должна себя вести: что подаём на вход, что должны получить, что считается ошибкой. Эта запись называется тестом, она проверяет код автоматически. Потом пишется код реализации. После этого код приводят в порядок, улучшают вид, не меняя функционал — это называется рефакторингом. Цикл повторяется десятки раз в день, маленькими шагами.



«ЕСЛИ БЫ
ПРОГРАММИСТЫ
СТРОИЛИ ДОМА»



**Почему же мы выбрали такой путь: работать по TDD?
Попробуем показать на примере комикса.**

Путешествие во времени: цена устойчивости

разработчик



клиент



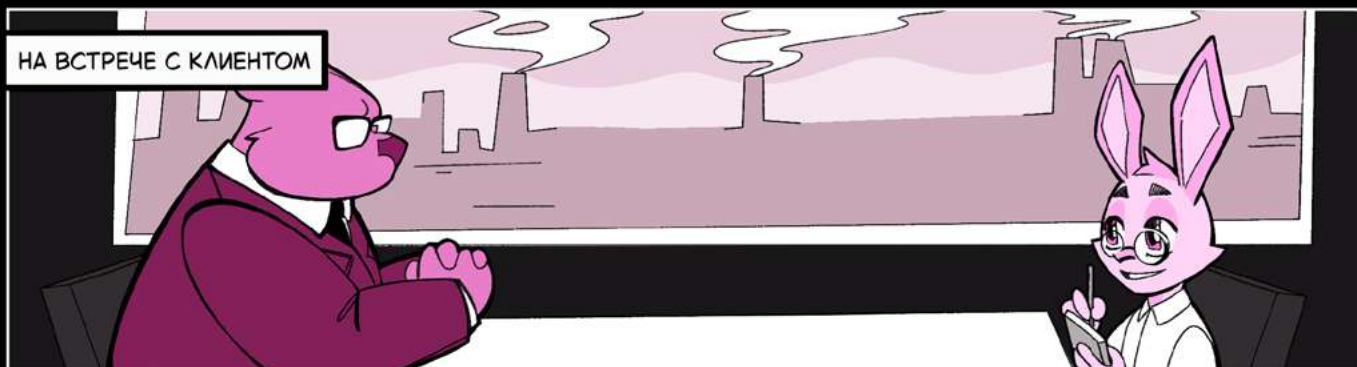
менеджер



менеджер



НА ВСТРЕЧЕ С КЛИЕНТОМ



У НАС МНОГО ПОДРЯДЧИКОВ.
ПОЧЕМУ МЫ ДОЛЖНЫ
ВЫБРАТЬ ВАС?



НУ, ЕСЛИ
КРАТКО...

ОЧЕНЬ МНОГО ГОВОРИТ О
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ГОВОРИТ ОЧЕНЬ МНОГО ГОВО
ОЧЕНЬ МНОГО ГОВОРИТОЧЕН
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ГОВОРИТ ОЧЕНЬ МНОГО ГОВО
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ГОВОРИТ ОЧЕНЬ МНОГО ГОВ
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН



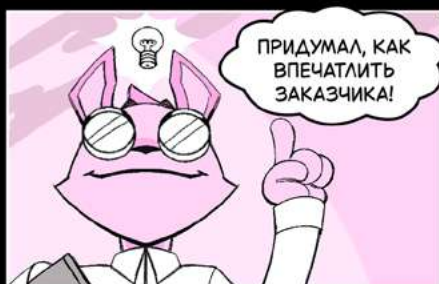
НАШЕЙ IT-СИСТЕМЕ
УЖЕ 12 ЛЕТ.
СДЕЛАТЬ ДЛЯ НЕЕ
МОДУЛЬ – БОЛЬШАЯ
ОТВЕТСТВЕННОСТЬ

ВАУ КАК КРУТО!
ХОЧУ ПРИЛОЖИТЬ
К ЭТОМУ РУКУ!

ОБСУДИМ С КОМАНДОЙ,
ЧТО МЫ МОЖЕМ
ВАМ ПРЕДЛОЖИТЬ,
И ВЕРНУСЬ



ПРИДУМАЛ, КАК
ВПЕЧАТАЛИТЬ
ЗАКАЗЧИКА!

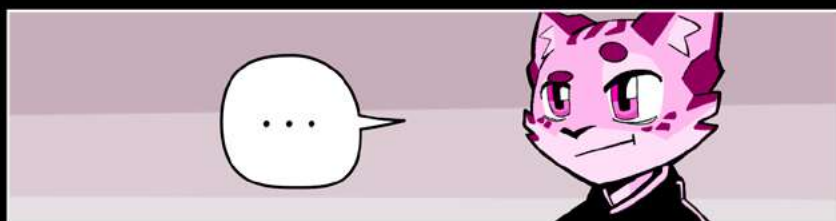


В ОФИСЕ

-20%
БЮДЖЕТА



КЛИЕНТ НЕ УВИДИТ ТЕСТЫ,
ЗАТО УВИДИТ ЭКОНОМИЮ.
УБИРАЕМ TDD, ОСТАВЛЯЕМ РЕЗУЛЬТАТ





ПРОХОДИТ 1 МЕСЯЦ.
СОЗВОН С ЗАКАЗЧИКОМ

КАК ХОРОШО, ЧТО
УСПЕЛИ ВОВРЕМЯ!

ПЕРЕДАЕМ НА ТЕСТИРОВАНИЕ
НАШЕМУ ОТДЕЛУ КАЧЕСТВА

НУ ВОТ!
ВСЕ ЖЕ ХОРОШО

ПРОСТО
СМОТРИ
ДАЛЬШЕ...

ПРОХОДИТ МЕСЯЦ

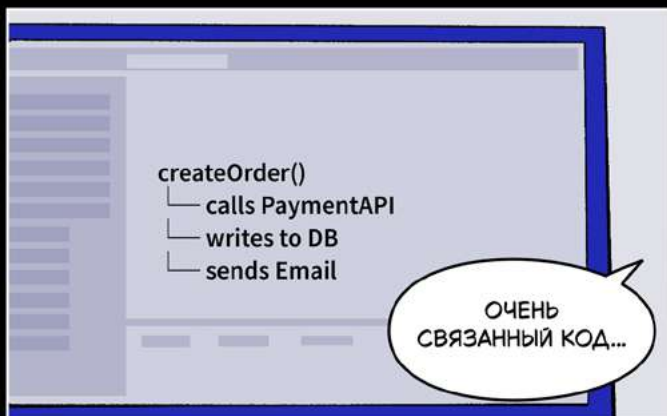
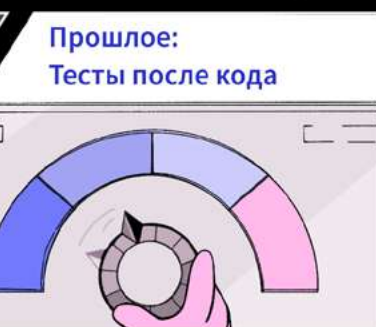
КЛИЕНТ:
ТАМ ЕЩЕ ОДИН БАГ НАШЛИ!

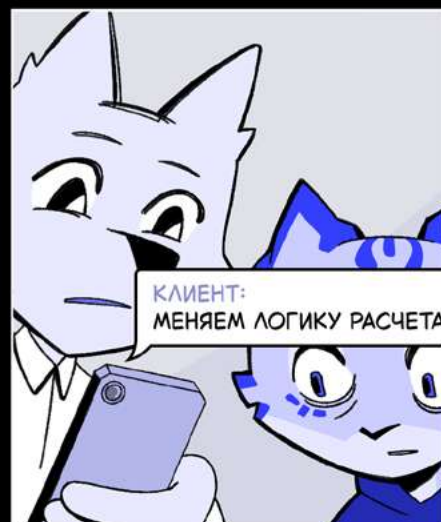
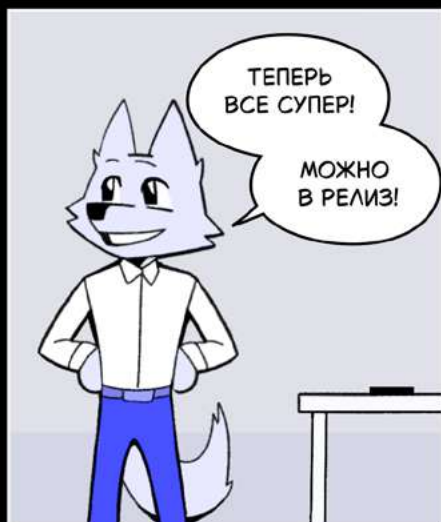
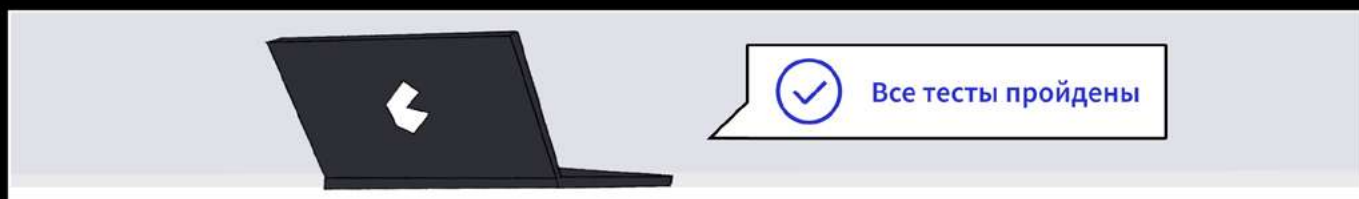
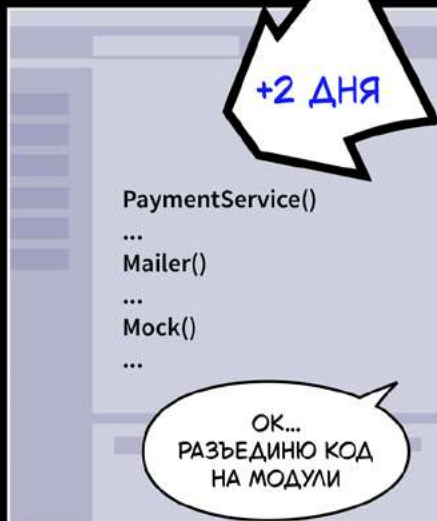
КЛИЕНТ:
ВЫ ПРЕДЫДУЩИЕ ЕЩЕ НЕ ЗАКРЫЛИ!!

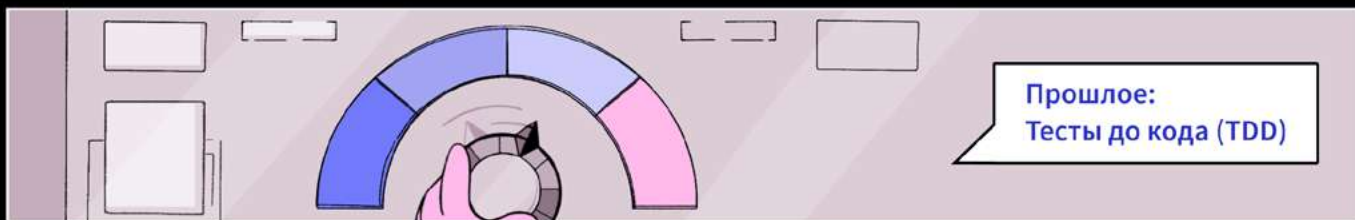
КЛИЕНТ:
КАК ВЫ МОГЛИ ТАК НАЛАЖАТЬ!

ПОТОМ ЕЩЕ МЕСЯЦ
РАЗГРЕБАЛИ.
И К РЕЛИЗУ НЕ УСПЕЛИ

И КЛИЕНТА ПОТЕРЯЛИ
— БОЛЬШЕ ОН НАМ ПРОЕКТ
НЕ ДАСТ







СНАЧАЛА ФИКСИРУЕМ
ЧТО ДОЛЖНО
ПРОИСХОДИТЬ



ВИДИШЬ? ОН СРАЗУ
ЗАДАЕТ ГРАНИЦЫ
МЕЖДУ ЧАСТЯМИ
СИСТЕМЫ. ТЕСТ
ЗАСТАВЛЯЕТ ЕГО
ПРОДУМЫВАТЬ
ДИЗАЙН КОДА СРАЗУ,
А НЕ ПОТОМ





Подход	Что получается	Когда оправдано
Без тестов	Быстро на старте, дорого потом	MVP. Прототип. R&D-эксперимент. Весь код, который готовы выкинуть, и начать писать заново.
Тесты после кода	Контроль есть, но дорого менять	Система с горизонтом 1–1,5 года. Стабильные требования.
TDD	Дороже сначала, дешевле на длинном го- ризонте	Системы с горизонтом от 2–3 лет. Активно меняющиеся требования. Высокая цена бага в проде.

Ручное тестирование как отдельную полноценную стратегию в таблицу мы не выносим. Оно не масштабируется на проекты с длинным горизонтом и частыми изменениями, и в большинстве случаев заменяется автотестами или точно дополняет их.

Это не только наш опыт

Когда мы говорим заказчику: «Давайте работать по TDD», — реакция предсказуема: звучит, конечно, красиво, но это же дольше и дороже. Да и как оценить эффект? Кто сказал, что будет качественнее?

В 2008 году исследователи из Microsoft Research, IBM и Университета штата Северной Каролины опубликовали результаты по чётвёртым промышленным командам. Три из них из Microsoft: это команды, разрабатывающие Windows, MSN и Visual Studio. Одна команда из IBM, которая разрабатывала драйверы для устройств.

Команды и продукты совершенно разные: разный объём кода (от 6 до 155 тысяч строк кода), разное время эксперимента (от 24 до 119 человеко-месяцев), разная доменная область проектов.

Эти команды работали по TDD. Рядом с ними работали команды с похожими проектами, но без применения TDD-подхода.

И вот результат:

Плотность дефектов у TDD-команд оказалась ниже на

40–90%

чем у сопоставимых команд без TDD.

За это пришлось заплатить временем: руководители оценили рост сроков начальной разработки в

15–35%

Важная деталь в том, что оценивались сроки начальной разработки. В этот срок не заложено дальнейшее сопровождение продукта. А именно там и утекает основное время: правки по багам, регрессионное тестирование продукта после любых изменений, разбор инцидентов, погружение новых участников проекта в кодовую базу.

У TDD-команд процессы сопровождения быстрее, потому что дефектов в коде меньше, тесты документируют поведение, а менять покрытый автотестами код можно без страха: автотесты покажут, если что-то сломалось.

То есть **15–35%** времени и бюджета — это цена, которую нужно заплатить один раз, в обмен на качество, которое работает дальше.

Рентабельность использования TDD-подхода можно оценить по времени жизни самого проекта: если проект будет жить долго, то вложения окупятся.



ИССЛЕДОВАНИЕ
MICROSOFT
и IBM

Противоречащее исследование

В 2021 году вышел метаанализ из 12 экспериментов. В этих экспериментах участвовали новички в программировании и опытные программисты, но ни у кого из них не было опыта работы с TDD. Задачи были учебными, и на них отводилось 2.5 часа. В этом исследовании TDD не показал преимущества над итеративной разработкой с короткими циклами с тестами после кода.

Это исследование не противоречит исследованию Microsoft и IBM, а добавляет к нему мысль: на дистанции в часы для людей без опыта с TDD эффект не успевает проявиться. Авторы также пишут, что нужны эксперименты с экспертами методологии на длинных задачах.

Мы работаем чаще с длинным горизонтом, поэтому ориентируемся на Microsoft и IBM.



ПРОТИВОРЕЧАЩЕЕ
ИССЛЕДОВАНИЕ

Семь условий, при которых TDD начинает работать

В 2011 году исследователи из Mälardalen University прошли обзором по работам про TDD в индустрии и задались вопросом: почему практика, которая в исследованиях даёт качество, в работе приживается так плохо? Обычно эту работу цитируют как «семь причин работать без TDD».

Мы переделали выводы из статьи в чек-лист из семи условий, при которых TDD начинает давать тот эффект, который зафиксировали Microsoft и IBM, и который мы можем подтвердить в своих командах.

1

Готовность платить за старт. Команда, руководители и заказчики заранее принимают, что в переходный к TDD-разработке период задачи будут занимать больше времени. Если между всеми участниками процесса нет согласия по этому пункту, под давлением сроков от TDD быстро откажутся.

2

Опыт работы по TDD у команды. Часто проблемы внедрения появляются от непонимания того, как это должно происходить. Начать закрывать эту базу можно с помощью книг основоположника TDD Кента Бека, обучающих видео и open source репозиториях.

- 3 **Навык писать тесты.** Плохо написанные тесты — это тесты хрупкие, дублирующие друг друга, проверяющие реализацию вместо поведения. С плохими тестами TDD превращается в дорогую, сложно поддерживаемую иллюзию контроля.
- 4 **Минимальный дизайн.** Даже при разработке по TDD, где тесты в том числе помогают проектировать дизайн кода, необходимо перед стартом представлять контуры разрабатываемого модуля: границы, интерфейсы, ответственность. Без этого тесты переписываются на каждом шагу.
- 5 **Дисциплина соблюдения цикла.** TDD работает только тогда, когда команда действительно придерживается этого подхода, а не имитирует его. На практике дисциплина в этом вопросе может «хромать» из-за нехватки времени или нежелания следовать подходу.
- 6 **Подходящая предметная область и инструменты.** Авторы статьи пишут про то, что в некоторых областях, например, тестировании интерфейсов, не хватает инструментов и необходимо их развитие. Эта статья 2011 года. С тех пор прошло много времени, и индустрия сделала скачок — для веб-интерфейсов появились, как минимум, Cypress и Playwright. Сейчас индустрия тестирования сильно повзрослела: инструменты можно найти и адаптировать в зависимости от предметной области.
- 7 **Не легаси.** На непокрытом унаследованном коде TDD в чистом виде не запускается. Сначала нужно отдельным усилием провести рефакторинг системы под новый подход, и потом запускать TDD в бой. Это самостоятельный проект, который стоит оценивать отдельно.

Хорошая новость в том, что ни одно из семи условий не упирается в магию или таланты «звёздной команды».

- Готовность платить за старт — вопрос договорённости с заказчиком.
- Опыт, навык писать тесты, дисциплина работы по TDD тренируются.
- Минимальный дизайн и подходящие инструменты закладываются на этапе оценки.
- Легаси выносятся в отдельный контур работ.

КАЖДОЕ ИЗ УСЛОВИЙ —
ЭТО ИНЖЕНЕРНАЯ ИЛИ УПРАВЛЕНЧЕСКАЯ
ЗАДАЧА, У КОТОРОЙ ЕСТЬ
РЕШЕНИЕ.



С чем можно сравнить TDD в других отраслях?

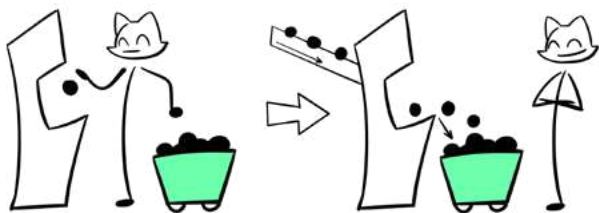
Toyota Production System (TPS) подарила нам не только крутые автомобили, но и своё видение того, как нужно строить процесс производства, чтобы контроль качества был вшит в работу, а не переносился на последний этап. Для этого в компании работают разные концепции, например, Lean (бережливое производство), Kanban (визуализация пути задачи), Кайдзен (философия маленьких шагов).

В контексте Test-Driven Development самые интересные принципы из видения Toyota – это Jidoka и Ji Kotei Kanketsu.

Jidoka (Дзидока)

Тойота описывает принцип дзидока как «автоматизацию с человеческим лицом». Его главные элементы: автоматическое обнаружение дефектов при производстве, немедленная остановка работы при обнаружении проблем и мудрое участие человека в их решении.

Сакити Тоёда начал формировать его задолго до его внедрения в производство. Отец будущего основателя групп компаний Toyota искал, чем облегчить работу матери, которая допоздна ткала на ручном станке. Он изобрёл автоматический станок, основной особенностью которого была не просто замена ручной работы: станок сам умел останавливать работу при обрыве нити. Эта способность станка помогла поднять и производительность, и эффективность — отсюда и пошла идея дзидоки.



На производстве автомобилей Toyota это трансформировалось в то, что автоматика делает всё, что можно доверить машине, а человек делает всё, что требует суждения. Станок замечает отклонение и останавливается, после чего приходит человек, оценивает, из-за чего произошла проблемная ситуация, и принимает решение о том, что нужно сделать сейчас.

Андон —

Это система, которая оповещает работников о проблемах с качеством. Начинается она со шнура, который нужно дёрнуть, чтобы остановить линию производства. Система или любой рабочий на линии, увидев проблему, дёргает шнур, производство останавливается, загорается табло, и к нему идёт лидер команды. Если лидер успевает решить проблему за такт — линия едет дальше, если нет — останавливается весь конвейер до момента устранения проблемы. Право остановить производство есть у каждого, не только у лидера команды или ответственного за качество. Система и сама умеет определять, что появился дефект, и останавливаться — но это не исключает человека, а лишь помогает снизить человеческий фактор в процессе контроля работы производства.



Сначала кажется, что остановка всего конвейера — это прямые потери для компании: работники и машины простаивают, план по выработке горит. Но на самом деле дефект, провезенный дальше по конвейеру, стоит дороже: его придется вынимать из уже собранного узла или механизма на более поздних этапах сборки, а если он доехал до дилера — то и вовсе из репутации компании.

Ji Kotei Kanketsu (JKK)

Ещё один принцип, который говорит про личную ответственность за полученный результат.

Ключевая идея: дефект нужно найти и устранить там, где он возник, потому что следующий за тобой процесс работы — это твой клиент, которому нельзя отправлять брак.

Каждый сотрудник обязан решать проблему в момент её возникновения. Важной составляющей этого принципа является стандартизация работы — четкое понимание и визуализация того, что должен делать каждый работник в общей системе. Без стандартизации следующий процесс никогда не знает, какую деталь для крепления он получит в этот раз.



Возвращаемся в разработку

И проводим аналогию. Тест, написанный до кода, — это и есть датчик обрыва нити, встроенный в станок, или датчик обнаружения дефектов на производственной линии. Такие датчики при работе по TDD разработчики составляют по всему продукту по ходу развития проекта.

Это не отдел контроля качества, который в конце производства партии принимает её и разводит руками, а механизм, который встроен в производство, и знает, как всё должно быть. Он не допускает неожиданного изменения формы колеса или развернутого в обратную сторону водительского кресла. Красный тест, который не прошел, — это андон. Он не просто сообщает, что где-то что-то случилось, он даже подсвечивает табло над конкретным местом появления дефекта.

Философский подход к работе у Toyota, основанный на нескольких важных принципах и автоматике, выходит за пределы производственных цехов. Фокус принципов работы на предотвращении дефектов на наиболее раннем этапе, анализ первопричин, стандартизация и расширение возможностей работников по вмешательству в рабочий процесс всей компании способствует формированию культуры ответственности за качество. Контроль качества, встроенный в процесс работы, — это не только про достижение результатов в краткосрочной перспективе, но и про создание системы мышления команды, которая стратегически обеспечивает успех работы компаний на длинном горизонте.

Анатомия теста: хороший против плохого

Среди условий, при которых TDD начинает работать, есть задачка со звёздочкой: «Навык писать тесты». Проще начать с малого: посмотреть на плохие тесты и на хорошие, и разобраться, что отличает одни от других.

```
def test_order():
    o = Order()
    o.add(Item("book", 500), 2)
    o.user = User(status="premium")
    o.recalculate_order_sum()

    assert o.total == 500 * 2 * (1 - PREMIUM_DISCOUNT)
    assert o.items_count == 2
    assert o.user.status == "premium"
    assert o.log[-1] == "recalculated"
```

Плохой тест

Что же с этим тестом не так?

- 1 Имя, которое не помогает сходу понять смысл сценария и проверки. `test_order` в лучшем случае может быть названием папки с тестами, но не названием конкретной проверки.
- 2 `total` вычисляется с помощью формулы, которая, скорее всего, точно в таком же виде существует внутри кода реализации. В чём тогда проверка?
- 3 В одном тесте четыре проверки про четыре разные вещи. Система проверки качества должна сигнализировать о том, что сломалось что-то конкретное, и сразу указывать на то, какой участок кода сломан. В текущей реализации не понятно: то ли сломан подсчет суммы, то ли статус пользователя не записывается, то ли количество позиций в заказе не учитывается. Такая проверка не помогает быстро найти место поломки и починить его.
- 4 Мы видим реальное устройство кода внутри: создаем заказ, добавляем в него элемент, видим, через какой метод происходит пересчет стоимости заказа. Нарушается инкапсуляция кода. Это проверка реализации, а не поведения. При малейшем изменении логики или даже рефакторинге кода реализации нужно будет переписывать также и эти тесты.

```
def test_premium_user_gets_10_percent_off_order_total():
    order = order_with(
        items=[
            ("book", 500),
            ("book", 500)
        ],
        user=premium_user(),
    )
    total = order.total()
    assert total == 900
```

А чем хорош этот тест?

- 1 **Имя.** Оно написано понятно — понятно что, понятно с кем, понятно когда происходит. Даже если вы на дейли-митинге скажете это название теста менеджеру, он поймет, что здесь проверяется.
- 2 **Никаких формул, повторяющих код реализации.** Если total равен 900, то он равен 900 — четкому числу, которое заранее известно по бизнес-правилам, а не формуле, которая может ошибиться или вовсе повторяется внутри реализации.
- 3 **Атомарность.** Один тест проверяет один смысл, не пытается проверять всё подряд. Это не значит, что assert должен быть один. Внутри одного теста может быть несколько проверок, но все они должны проверять один механизм. Три assert'а, описывающие одно поведение — это нормально. Три assert'а, описывающие разные механизмы — нет.
- 4 **Тест ничего не знает о том, как именно заказ считает сумму.** Формирование заказа спрятано за фабрикой. Внутри реализации можно менять всё, рефакторить, а тест продолжит оставаться зелёным, потому что он изначально покрывал бизнес-сценарий. Тест проверяет поведение, а не реализацию.

*Хороший тест
переживёт
рефакторинг,
плохой придётся
менять вместе
с реализацией.*

Какие ещё есть сигналы о том, что что-то не так?

- 1 Тест падает через раз. Такие тесты называют flaky — флакающими. Сегодня он зелёный, завтра красный, хотя по его коду ничего не менялось. Часто виноваты состояния гонок, случайно сгенерированные данные, зависимость от времени или переиспользуемый между тестами объект, который поменялся в одном тесте, а в этом используется с ожиданием, что он старый. Такой тест хуже отсутствующего, потому что команда быстро учится перезапускать тесты, дожидаясь успешной попытки, вместо того, чтобы реагировать на падение, искать ошибку и исправлять.
- 2 На то, пройдет тест или нет, влияет порядок его запуска в общем списке. Не надо так. Тесты не должны зависеть друг от друга. Тогда они не будут флакать, а ещё их можно будет запускать параллельно и ускорять время прогона тестирования.
- 3 Внутри теста есть логика. Если в тесте есть условия, циклы, обработка ошибок или формулы — это уже признак того, что тест сам нуждается в тесте. Проверка должна быть настолько простой, чтобы её правильность была очевидна с первого взгляда.
- 4 Тест ни разу не падал. Если вы «ломаете» строку в коде, а тест остаётся зелёным, это значит, что тест никогда ничего не сторожил. Поэтому при TDD важно, чтобы после написания теста вы сначала его запускали и видели, что он упал. И только после добавления реализации проходил успешно.
- 5 Подготовка к тесту длиннее проверки в десять раз. Сорок строк подготовки объектов для одного assert — это часто маркер того, что в коде не хватает инструментов для написания хороших и простых тестов.
- 6 Тест невозможно назвать. Если не получается придумать имя теста осознанно и выходит что-то вроде «test_process_data_2», значит, у проверяемого куска нет внятного поведения или смысла.
- 7 Над тестом стоит комментарий, объясняющий, что он делает. Это значит или то, что у него плохое название, или то, что его сложно читать. И то, и другое его не красит.

Плохое имя и лишние проверки можно найти глазами сразу. Остальные сигналы становятся заметны во время жизни тестов и их сопровождения.

Про Arrange-Act-Assert и Given-When-Then

Все хорошие тесты похожи друг на друга, каждый плохой тест плох по-своему.

Что можно точно выделить у хороших тестов — это структуру. Такие тесты делятся на три части:



У этой структуры есть имя: её называют Arrange-Act-Assert — Подготовка-Действие-Проверка.

В BDD (Behavior-Driven Development, разработка через поведение) такую структуру называют Given-When-Then — Дано-Когда-Тогда. Формулу придумали Дэн Норт и Крис Мэттс в 2004-м: она была нужна, чтобы описывать поведение системы словами, понятными всем специалистам в команде. Язык написания сценариев, который превращает эту формулу в текст, называется Gherkin.

Сценарий в нём выглядит так:

Сценарий: Премиум-пользователь получает 10% скидки на заказ

Дано пользователь со статусом «премиум»

И заказ на 1000 рублей

Когда система считает итоговую сумму

Тогда сумма заказа равна 900 рублям

Синтаксис Gherkin и написание тестов на нем разворачивает мысль «хорошее имя теста можно произнести на дейли, и менеджер поймет» до конца: тест-сценарий сразу написан словами, которыми в компании говорят о продукте.

Не всегда сам синтаксис подходит для написания тестов в продукте, он имеет свои ограничения. Но что точно можно позаимствовать для своих тестов — это структуру. И если из теста нельзя с легкостью эту структуру выделить, подготовка перемешивается с действием, действие перемешивается с проверкой, действий несколько — значит, тест стоит переосмыслить. У теста с внятной формой есть словесная версия, которую не нужно переводить.

Книжная полка по тестированию



Разобранные в статье критерии хороших и плохих тестов — это малая часть кунг-фу, которое люди осваивают годами. Сократить этот путь поможет опыт других тестировщиков, описанный в книгах.

- 1 «**Экстремальное программирование. Разработка через тестирование**» — Кент Бек.

Главная книга по теме от автора концепции TDD. Содержит разбор реальных проектов и каталог приёмов.

- 2 «**Тест-дизайн. Практическое руководство для начинающих**» — Ольга Назина.

Рассказывает, что проверять, а не как оформить проверку. Классы эквивалентности, граничные значения, pairwise и готовые чек-листы.

- 3 «**Эффективная работа с унаследованным кодом**» — Майкл Физерс.

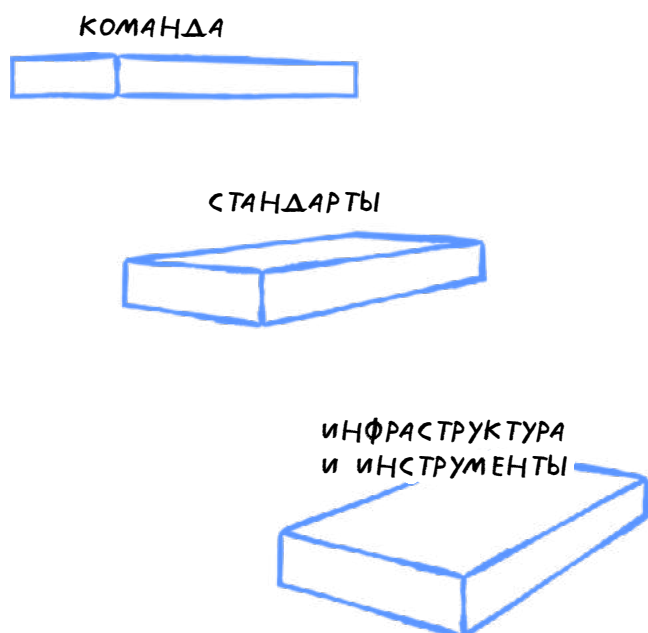
Что делать, когда красиво начать с нуля уже нельзя. Приёмы, которые помогают обложить тестами код, писавшийся без них.

Три опоры TDD

Чтобы подход с написанием тестов до кода начал работать, недостаточно просто выбрать правильную библиотеку, скомандовать разработчикам делать теперь только так и завтра начать. Это просто не сможет работать. Тесты до кода — это не инструмент, который можно добавить и забыть. Это процесс, который нуждается в поддержке и контроле. Если ничем не поддерживать, команда через пару недель вернется к написанию тестов после кода, если будет оставаться время. А времени всегда не хватает.

Тем более, что на время переходного периода разработка замедлится. Замедление заметно сразу, а качественный скачок станет виден не раньше, чем через квартал.

Чтобы пережить время перехода, увидеть первые результаты и получить силы на движение дальше, TDD должен поддерживаться на разных уровнях:



Сначала — представить результат. Потом — начать его делать. Это и есть TDD, и внедряется он ровно по тому же правилу: сначала — спроектировать внедрение процесса, потом — внедрить его.

На уровне инфраструктуры и инструментов:

Должно быть всё, чтобы тесты запускались. Звучит банально, но не всегда всё для этого готово. Выбрать фреймворки для написания тестов. Настроить запуск тестов на локальном окружении и на production. Оставить только те тесты из существующей базы, которые стабильно проходят, а которые проходят нестабильно — исправить или удалить.

Добавлять тесты должно быть легко, это не должно ощущаться как мучение. Запускать тесты тоже должно быть легко и быстро: если прогон тестов занимает десятки минут, то TDD-цикл будет очень длинным, контекст у разработчика будет переключаться, и возврат к новому TDD-циклу будет занимать больше времени.

На уровне стандартов:

Стандартизация помогает разным участникам команды следовать одним и тем же шагам и получать похожий результат. Как рецепт по шагам описывает, что делать с пирогом, так тестовая стратегия отвечает, какие тесты и на каком уровне писать при разработке системы. Все случаи в стандарт не поместятся, да это и не нужно: тестирование — это не только ремесло, но и искусство, и не все кейсы получится тестировать одинаково. Но базовые create-read-update-delete сценарии нужно рассчитывать увидеть одинаковыми у любого автора в проекте: та же структура, те же правила именования, предсказуемое место, где искать проверку.

На уровне команды:

Команда должна осознавать ценность подхода. Если верить, что всё получится, не выходит — достаточно гибкости попробовать новое как эксперимент: на одном модуле, на оговорённый срок, с заранее определёнными критериями успеха. Лидер следит за дисциплиной, участники следуют подходу, а не саботируют его. Хотя саботаж почти никогда не злая воля — чаще это сигнал, что где-то писать тест слишком сложно или не понятно как. В таком случае нужно или чинить инфраструктуру, или додумывать стратегию тестирования совместно с командой и QA-инженером.

Бинго: Можно. А зачем?

Возражения против тестов до кода.

Скоро релиз, не до этого	У нас есть для этого тестер	У нас легаси	Тут двадцать строк всего	Я не знаю заранее, каким будет интерфейс
С тестами дольше в два раза	Я не тестировщик	Это фронт, зачем его тестить	Это же просто CRUD	Мне неудобно так думать
Тесты в спринт не влезают	Я лучше потом сгенерирую тесты нейронкой	У нас ML, там непредсказуемо	Я руками проверил, всё ок	А какая разница: до или после?
Это MVP, мы потом перепишем	На стейдже поймают	У нас всё завязано на внешний сервис	У нас проект слишком большой	Я так и так одинаково напишу

20 фраз, но всего 6 разных возражений.
Найдите по цвету тот, который откликнулся вам больше всего:

«Не сейчас»

Не меряйте время до коммита. Измеряйте время до релиза, со всеми возвратами с ревью, багфиксами и отладкой чужого кода через месяц. Окажется, что счёт другой.

«Тут не нужно»

Решает не размер кода, не его сложность, а время его жизни. Чем дольше он живет, тем важнее проверять при любых изменениях кодовой базы, что он работает так же, как раньше.

«Тут не проверишь»

Сегодня нельзя проверить — завтра страшно поменять. Это техдолг, который надо решать, а не закрывать на него глаза. Всегда можно что-то придумать.

«Разницы нет»

Тест, написанный после кода, подстроен под написанный код. Красным вы его не видели ни разу — точно уверены, что он умеет падать?

«Проверит другой»

Тестировщик с нейронкой на стейдже смотрят на код и гадают, то ли он делает. Что на самом деле он должен делать, знает только автор (если не забыл).

«Я так не могу»

Неудобно формулировать — значит, задача ещё не ясна. Не знаете, каким будет интерфейс — тест его поможет придумать.

Поговорим про стандартизацию — и начнём с фронтенда

TDD-подход принято показывать на задачах с чёткой логикой, больше напоминающих core-логику на бэкенде: калькуляторы счёта игры в боулинг, валидатор паролей или расчёт стоимости корзины интернет-магазина. После таких примеров остаётся ощущение, что TDD — это что-то для учебных задач. Большое спасибо, что показали, но делать нам с этим нечего.

В Tourmaline Core Test-Driven Development начался, как ни странно, с фронтенда, а потом уже перетёк в другие направления. Давайте не будем смотреть в начало этого пути: там мы мучительно сопоставляем разные ситуации друг с другом и пытаемся понять, как будет выглядеть наша базовая тестовая стратегия. Посмотрим сразу в настоящее: какой процесс работы помогает нам быстро программировать с тестами сейчас.

Давайте на примере. Представим, что на нашем фронтенде появляется новая изолированная секция. На ней есть кружочек с аватаркой пользователя, его имя и количество непрочитанных сообщений.



Иванов Иван

4

Теперь работаем по шагам:

1 Согласовываем контракт между фронтендом и бэкендом.

Это позволяет нам разрабатывать фронтенд в своей песочнице, независимо от бэкенда, и быть уверенными, что в итоге части системы соединятся как надо.

Договариваемся с бэкендером, что данные для этой секции будут возвращаться по такому контракту:

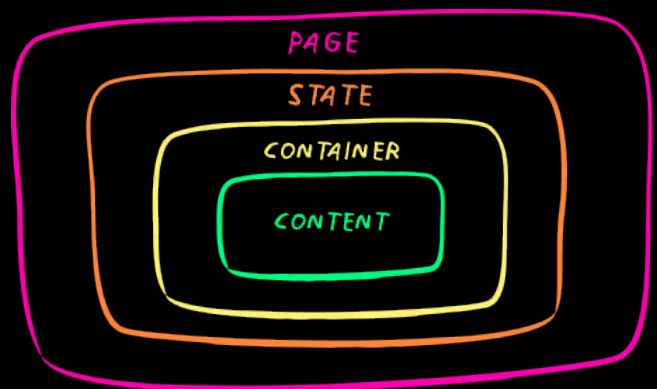
```
GET /user-info
{
  userPictureUrl: string,
  userFullName: string,
  numberOfUnreadMessages: number
}
```

2 Базово проектируем архитектуру.

Продумываем иерархию компонентов, слои, минимальный дизайн кода перед тем, как начать создание тестов.

Мы раскладываем страницу на четыре уровня: Page оркестрирует секции, State хранит данные и бизнес-логику, Container отвечает за сетевые вызовы, Content — за рендер и интерактив. У изолированных секций свои Container, State и Content.

Каждый слой не налезает на чужую зону ответственности: State ничего не знает про сеть, а Container ничего не рендерит. Эта нарезка одинакова на всех проектах — разработчик с одного проекта понимает структуру другого без онбординга.



В репозитории эта секция будет выглядеть так:

```
✓ features / layout / sections / user-info
  ✓ state
    TS UserInfoState.ts
    ⚙️ UserInfoStateContext.tsx
    ⚙️ UserInfoContainer.tsx
    ⚙️ UserInfoContent.tsx
```

3 Начинаем с базового сценария.

На фронтенде самая дорогая поломка — это та, которая мешает пользователю пройти ключевой бизнес-сценарий. E2E-тесты, которые покрывают ключевые пользовательские сценарии, ловят такие поломки лучше всего.

Придумываем happy path use case.

Сейчас он будет звучать так: «Я открываю главную страницу и вижу свои имя, фотографию и количество непрочитанных сообщений.» Случаи, когда что-то идет не так, здесь не учитываем — это задача других тестов.

Переносим use case в E2E-тест.

Пишем тест на той библиотеке, которую выбрали. Запускаем тест и видим, что он красный. Это правильно — реализацию мы ещё не писали. Тестом мы представили и зафиксировали наш будущий результат.

Написать реализацию.

Кодим, опираясь на ту архитектуру, которую задумали. Пишем код пока без учёта краевых случаев: как будто в мире не существует обработки ошибок, запросы не падают по тайм-ауту, а бэкенд всегда возвращает код 200. Запускаем тест. Доводим тест до позеленения: это будет значить, что ожидание совпало с реальностью, и мы попали в прогнозируемую точку.

4 Продолжаем с краевыми кейсами.

Задумываемся, что в базовом сценарии может пойти не так?

Работаем над слоем Content.

Как выглядит контент, когда нет непрочитанных сообщений? Как мы отрисовываем случай, когда у пользователя нет аватарки, рисуем ли заглушку? Для всех вопросов пишем тест, который фиксирует ожидаемое поведение, запускаем его, убеждаемся, что изначально тест падает, а потом добавляем минимальную реализацию, которая необходима, чтобы тест прошёл успешно.

Работаем над слоем State.

Допустим, на мобилке имя человека должно быть не полным, а сокращенным: вместо «Иванов Иван Иванович» — «Иванов И.И.» Как будет работать эта логика? Или, например, по задумке дизайнера, если уведомлений больше 99, то писать не полное число, а «99+». Фиксируем все эти случаи тестами и дописываем нужную реализацию.

Работаем над слоем Container.

Что будет, если при попытке загрузить данные выпадет ошибка? Показывается ли у нас лоадер во время загрузки данных по сети? Продумываем ответы на вопросы в тестах и дописываем нужный код.

Посмотреть воркшоп, как это делать на React, можно по ссылке:



ЗАПИСЬ
ВОРКШОПА



Александр, руководитель Tourmaline Core. Доклад на DevOops.

Вывод

Слои подсказывают нам, о чём нужно подумать, и заставляют зафиксировать ожидаемое поведение по всем вопросам. При ревью такого кода по тестам и описанным use cases проще понять, какие ситуации были тщательно отработаны, а какие могли быть забыты. По многим типам компонентов можно зафиксировать чеклисты проверок на уровне компании: простые компоненты должны делаться единообразно разными авторами.

Интерфейс пиксель-в-пиксель как в дизайне: вёрстка по TDD

Есть проекты, где идеальность интерфейса — это часть общего впечатления премиального продукта. У нас, например, была ситуация, когда заказчик под микроскопом (это не метафора, правда прямо под микроскопом!) проверял, что дизайн макета попиксельно совпадает с тем, что разработчики сделали на экране устройства в кнопке с градиентом. Мы дополировали тот градиент до идеала.

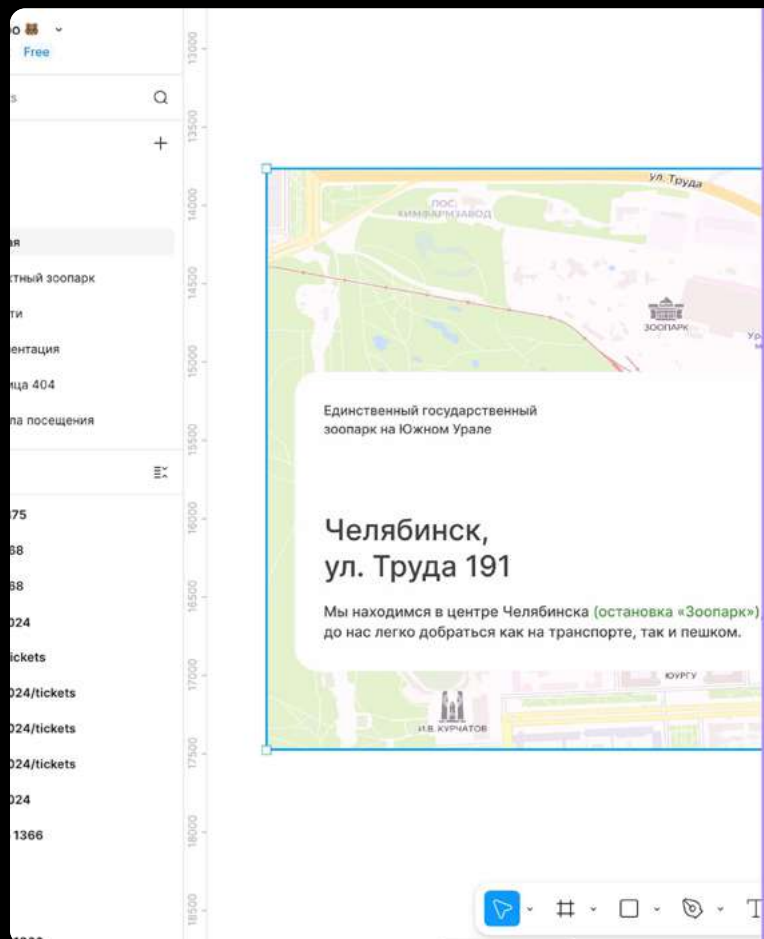
Ещё один пример, когда стоит внимательно проверять вёрстку — это компоненты дизайн-системы.

Дизайн-система переиспользуется между разными интерфейсами: один и тот же вид кнопок, чек-боксов, инпутов повсюду. Задача команды разработки, которая отвечает за дизайн-систему — сверстать её точно такой, какой её используют дизайнеры в макетах. Если сделать не так, то есть риск, что у дизайнера в макете будет красиво, а при разработке всё поедет из-за пары пикселей или смены цвета у элемента.



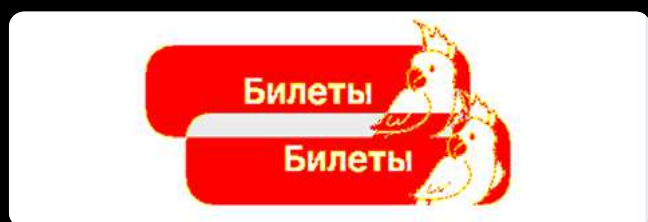
Вдруг в одном месте добавили кнопку, а разъехался интерфейс из-за этого в другом?

Раньше соответствие дизайну было зоной ручной проверки: дизайн-макет сравнивали глазами с открытым сайтом. Потом при вёрстке веб-страниц в браузере стали использовать плагин, который умеет накладывать полупрозрачную картинку макета на сайт. Сверять стало удобнее, но всё ещё это делалось вручную. Да и интерфейсы существуют не только в браузере — они есть в банкоматах, мобильных приложениях, телевизорах, на умных часах: везде, где есть дисплей. Наложить на них макет плагином так просто может не получиться. А если количество страниц, вкладок, всплывающих окон на сайте растёт, всё больше времени надо тратить на ручную визуальную проверку.



С появлением инструментов тестирования (для веба, например, это может быть Cypress или Playwright) скриншотные проверки можно автоматизировать и запускать при каждом изменении в вёрстке — один пайплайн на пару минут, и можно быть уверенным, что весь интерфейс в порядке.

Для этого нужно только две картинки — эталонное изображение от дизайнера и скриншот того, что получилось при разработке. Инструмент накладывает изображения друг на друга и сверяет каждый пиксель. Не совпавшие пиксели загораются красным. Это значит, что вёрстка больше не точно такая, как было согласовано с дизайнером.



Расхождение при скриншотном тестировании



Сайт Челябинского зоопарка

Когда мы говорим, что работаем по TDD, заказчики просят показать примеры. На коммерческих проектах это невозможно, потому что есть соглашение о неразглашении. Но у нас есть проект для заказчика с открытым исходным кодом — это сайт Челябинского зоопарка. Здесь мы оторвались по полной: протестировали всё, в том числе и вёрстку.

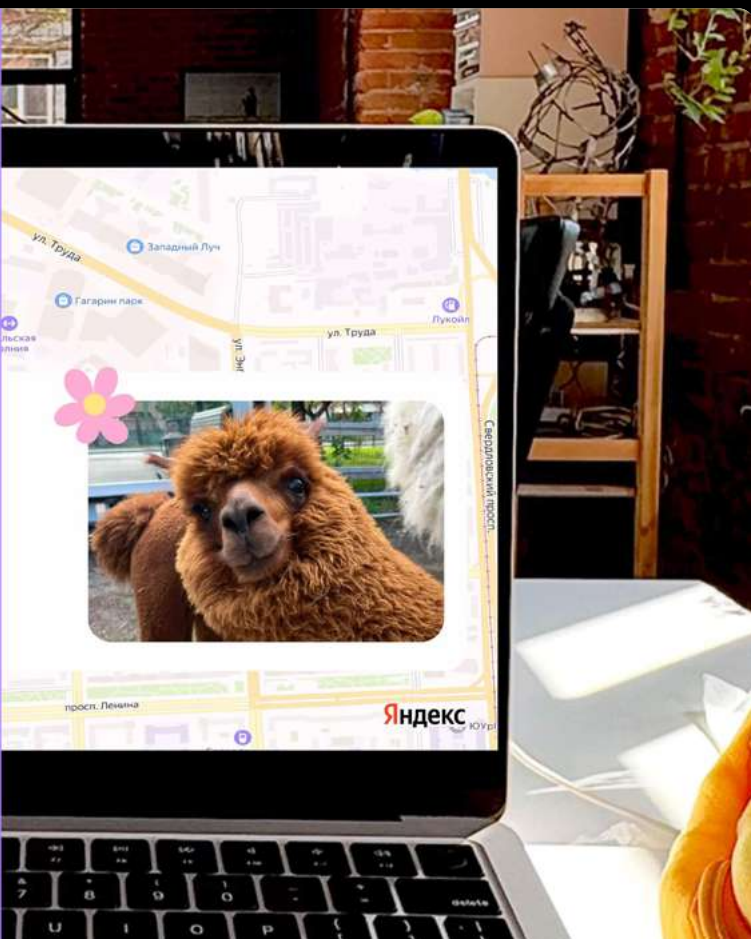
Главным вопросом было:

КАК ДЕЛАТЬ
ВЁРСТКУ ПО TDD?

Ответ — через скриншотные тесты.



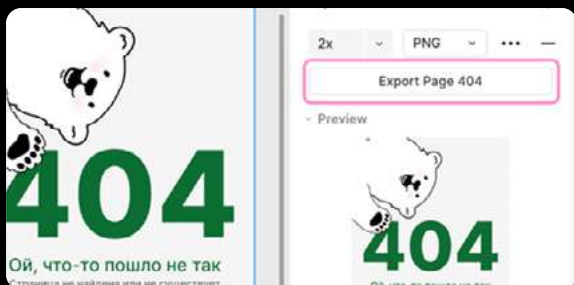
КЕЙС САЙТА
ЗООПАРКА



Вот как шла работа над вёрсткой сайта по TDD:

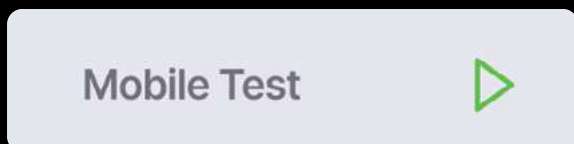
- 1 Экспортируем изображение компонента или экрана для всех разрешений из Figma.

Мы традиционно делаем дизайн для мобилки, планшета, десктопа и широкого экрана. Эти изображения — спецификация того, какой должна получиться вёрстка.



- 2 Оборачиваем изображения в автотест.

Чтобы запускать автоматическую сверку скриншотов с тем, что навестают разработчики.



- 3 Верстаем по макету.

Достигаем максимума сходства с изображениями из Figma.

- 4 Отправляем на приёмку дизайнеру.

Макет с вёрсткой совпадают не всегда. Иногда они различаются в пиксель, который стилями не поправишь. Тогда показываем дизайнеру расхождение и спрашиваем: «Норм?». После уверенного «Да!» генерируем новое эталонное изображение по вёрстке. Теперь используем его для скриншотных тестов. До приёмки тест проверяет соответствие дизайну, после приёмки — стабильность результата.

- 5 Запускаем тест.

И видим, что он зелёный. Больше менять вёрстку в проекте не страшно — красные пиксели в тестах подскажут, если что-то пойдет не так.



Екатерина и Артём, фронтенд-разработчики Tourmaline Core. Запись доклада.



ЗАПИСЬ ДОКЛАДА
ПРО ВЁРСТКУ
ПО TDD

В мобилке и на устройствах механика для тестов та же, меняется только инструмент, с помощью которого пишутся тесты. Главное, что инструмент должен уметь делать — это накладывать изображения друг на друга и находить несоответствия.

Заметка про рефакторинг

Почему-то, когда показывают TDD, шаг рефакторинга пропускают. Раз-раз и готово, почти сразу чистовой код. Такое ощущение, что черновики никто не пишет.

Но, конечно, в реальной жизни без него никак. Перед стартом TDD-цикла мы делаем базовое проектирование, а что выйдет в итоге, детализируем уже по ходу, в том числе с помощью тестов.

И можно страшно отчаяться, выяснив по ходу разработки, что архитектуру нужно сделать другой: часть кода вынести в отдельный класс, часть вовсе удалить из метода. Если тесты гвоздями прибиты к реализации, то и их тоже придется переделывать. Отсюда растет ощущение, что TDD мешает и проще его выкинуть.

Поэтому тесты стоит держать повыше, на поведении. Тогда черновик можно менять сколько угодно, тесты этого не заметят. Дальше в номере есть пример: на бэкенде мы меняем словарь в памяти на настоящую базу, переписываем внутренку эндпоинтов целиком, но не трогаем в E2E-тесте ни строчки.

За кулисами контракта: делаем бэкенд

Канонические примеры не помогают понять разработку через тестирование не только на фронтенде, но и на бэкенде тоже: посмотрел на то, как тестировать сложные функции, а потом открыл свой бэкенд и увидел там обращения к базе и запросы к другим сервисам по HTTP. Остается недоумение: а что делать-то?

Представим, что мы пишем бэкенд для работы с пользователями и реализуем create-read-update-delete эндпоинты.

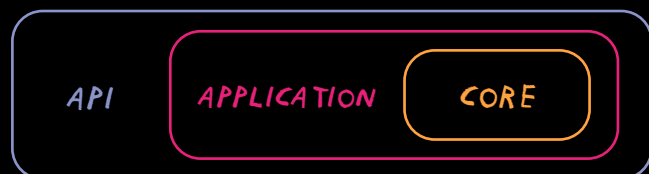
Вот что будем делать:

1 Контракт между фронтендом и бэкендом.

Договариваемся о том, как будут выглядеть эндпоинты для обмена данными. После этого объявляем эндпоинты в коде бэкенда со всеми моделями запросов и ответов, но пока что не пишем реализацию — можно добавить заглушку.

2 Проектируем архитектуру.

Мы раскладываем бэкенд на луковую архитектуру с такими слоями: API, Application, Core.



API — это тонкий слой. Он принимает запрос, выполняет все декораторы (middlewares), например, проверяет наличие пермиссий у аккаунта, и делегирует обработку запроса в следующий слой.

Application реализует все сценарии: работает с базой данных, превращает сущности в модели ответов, делает запросы к соседним сервисам. Здесь проходит основная работа. Код внутри Application нарезан по фичам: всё для создания пользователя лежит в одной папке — запрос, ответ, команда, обработчик запроса и тесты к ним.

Core держит доменные сущности и выполняет бизнес-расчёты.

Общее правило: слой не может вызывать наружные слои, только внутренние. Слои не налезают друг на друга и не делают работу своих соседей.

Так это будет выглядеть в репозитории, когда мы закончим (на примере API на .NET):

```

  ▾ Api/Features/Users/
    • UsersController.cs
    • UsersControllerTests.cs
  ▾ Application/Features/Users/Handlers/
    CreateUser/
    • CreateUserCommand.cs
    • CreateUserCommandTests.cs
    • CreateUserHandler.cs
    • CreateUserHandlerTests.cs
    • CreateUserResponse.cs
    • CreateUserRequest.cs
  ▾ Core/Features/Users/Entities
    • User.cs
    • UserTests.cs
```

Тесты мы кладём рядом с кодом, не делая отдельную зеркальную папку с дублированием структуры: когда-то нас покусал фронтенд, и теперь нам кажется более удобным видеть файлы тестов рядом.

3 Начинаем с happy path сценария.

Набрасываем код реализации, отталиваясь от happy path — когда всё идёт по плану и не происходит внештатных ситуаций.

Придумываем use case.

Получается такой: «Я создаю пользователя, читаю его, обновляю имя и удаляю».

Пишем E2E-тест на happy path.

Базовое правило: надо, чтобы каждый эндпоинт попал в E2E-тест хотя бы один раз. Создаём пользователя через эндпоинт, получаем всех пользователей и проверяем, что тот самый есть в общем списке, обновляем имя, запрашиваем именно его и видим обновления с прошлого шага, после чего удаляем этого пользователя. Теперь в общем списке на запрос его нет. Случаи, когда что-то идет не так, и происходят exceptions — это не зона ответственности этих тестов. E2E отвечают за всё хорошее и верхнеуровневое в этом мире. Мы их пишем с помощью инструмента Karate и используем из него декларативный синтаксис языка Gherkin. Эти тесты не зависят от языка бэкенда: они выглядят одинаково и для .NET, и для Python, и для C++ бэкендов. Запускаем тест — он красный, потому что внутри пока заглушка. Тестом мы зафиксировали будущий результат.

Пишем реализацию, распределяем код по слоям.

И здесь есть такой приём: чтобы заставить E2E-тест проходить успешно, база данных пока не нужна. Первая реализация может хранить пользователей в словаре в памяти приложения. Мартин Фаулер называет такое фейком (fake) — работающая реализация, которая срезает углы и не годится для продакшена. Кодим без учёта краевых случаев, пишем для этого минимальное количество кода: наша задача сейчас только в том, чтобы прошёл E2E happy path.

Заменяем фейк на настоящую базу.

Появляются сущности, работа с ORM (Object-Relational Mapping), миграции, команды и запросы. Мы переписали всю внутренность эндпоинтов — и не тронули ни строчки в E2E-тесте. Он все еще зелёный, потому что проверяет поведение, а не реализацию. Это и есть сила E2E — этим тестам совершенно всё равно, на каких инструментах написана реализация внутри и сколько там слоёв.

4 Прорабатываем краевые кейсы и распределяем тесты в соответствии с архитектурой.

Core — тесты почти без моков.

В Core в основном хранится доменная логика, и чаще всего это чистые функции. Допустим, в Core мы в функции складываем фамилию, имя и отчество в одну строку. Тут можно задаться такими вопросами: что будет, если отчества нет? А если фамилия из двух слов через дефис? Каждый вопрос — тест со входом и выходом, без базы и без HTTP.

Запросы и команды — тесты с реальной базой.

Это не логика на C#, это то, что произойдёт в SQL: фильтрация, сортировка, группировка, и проверять такое на списках в памяти бессмысленно. Мы поднимаем базу в Docker, кладём туда трёх пользователей и проверяем, что вернулись нужные два при соответствующих фильтрах. Каждый тест запускается в своей транзакции с уровнем изоляции Serializable, и после прохождения теста транзакция откатывается, поэтому тесты не мешают друг другу. Ещё вариант, когда нужны эти тесты — это когда часть бизнес-правил реализована средствами базы: например, уникальность почты пользователя гарантируется соответствующим индексом.

Хендлеры — тест на сценарий целиком.

Хендлеры отвечают за оркестрацию работы с командами, запросами и внешними сервисами. Также именно в хендлерах происходит маппинг ответов API. Здесь мы тестируем поведение, все ответвления и случаи, когда что-то отвалилось, например, упал внешний сервис.

Контроллеры — интеграционный тест через HTTP.

Поднимаем приложение и стучимся в эндпоинт. Проверяем не бизнес-логику, а валидацию контракта: что при пустом обязательном поле придёт 400, а в теле будет ожидаемая форма ошибки. На этом слое могут понадобиться моки, чтобы эмулировать ответы от других сервисов, можно даже отключать работу с базой, если нужно.

Покрывание каждого типа тестов считается отдельно: юнитов, интеграционных, E2E. В README висит не один бейдж общего покрытия, а четыре. Разница между ними говорит больше, чем общая цифра: видно не только сколько кода покрыто, но и чем именно, и так можно быстро понять, какое перекрытие сейчас между слоями тестирования.



СЕРИЯ ВИДЕО

ПРО TDD

НА БЭКЕНДЕ

Когда мы начинаем от happy path E2E-теста, а потом переходим к детализации краевых кейсов на разных слоях — это частичка итеративного подхода к разработке. Это можно сравнить с рисованием портрета. Можно двигаться инкрементально: например, сначала тщательно прорисовать глаза, а потом перейти к бровям и тщательно проработать их. А можно двигаться итеративно: сначала набросать на скорую руку полный образ человека, а потом прорисовывать везде по чуть-чуть, тем самым постепенно проявляя картину. В случае с тестами и реализацией новой фичи также: мы сначала набрасываем общую композицию и всего понемногу через E2E-тест и реализацию happy path, а потом начинаем постепенно детализировать на разных слоях краевые кейсы.

Через такой TDD-цикл наша картина проявляется ритмично.

Щьём железо: добавляем TDD в embedded

TDD во встраиваемой разработке кажется сложной темой

Это же значит, что нужно часто писать тесты и много раз менять реализацию — а значит, часто билдить прошивку и заливать её на устройство для запуска тестов. На некоторых долгоживущих и развивающихся системах полная сборка, прошивка и ожидание отчёта занимают >10 минут. То есть шаг запуска тестов растягивается до таких размеров, при которых использование TDD — плохая идея.

Тестировать код только локально плохо: какие-то проблемы могут проявиться уже на платформе устройства, если его архитектура отличается от архитектуры рабочей станции разработчика (а так почти всегда и бывает). Char может быть знаковым или беззнаковым в зависимости от архитектуры и компилятора. Указатель на хосте занимает 8 байт, на 32-битном таргете 4. Стек на хосте мегабайты, на таргете килобайты. Бац! И у вас баг, который локально не воспроизводится.

Так можно ли сократить время TDD-цикла в embedded-разработке?

Можно попробовать с техникой **Dual Targeting**. Это когда один и тот же код собирается под две целевые платформы: хост и таргет. Хост — это локальная машина разработчика, а таргет — платформа, на которой работает целевое устройство. То есть кодовая база у нас одна, а тулчейнов и сборок — по две.

Архитектура строится вокруг трёх уровней:

1 Логика — конечные автоматы, расчёты, планировщики.

Она не зависит от устройства, поэтому может тестироваться локально, а на устройстве проходить финальную верификацию. Этот уровень можно разрабатывать по TDD на хосте.

2 Порты — узкие интерфейсы.

«Отправь байт», «поставь таймер», «прочитай память». На хосте добавляем фейковые реализации, которые помогут изолировать разработку логики от устройства, а на таргете используем настоящие драйверы.

3 Драйверы — связывают порты с железом.

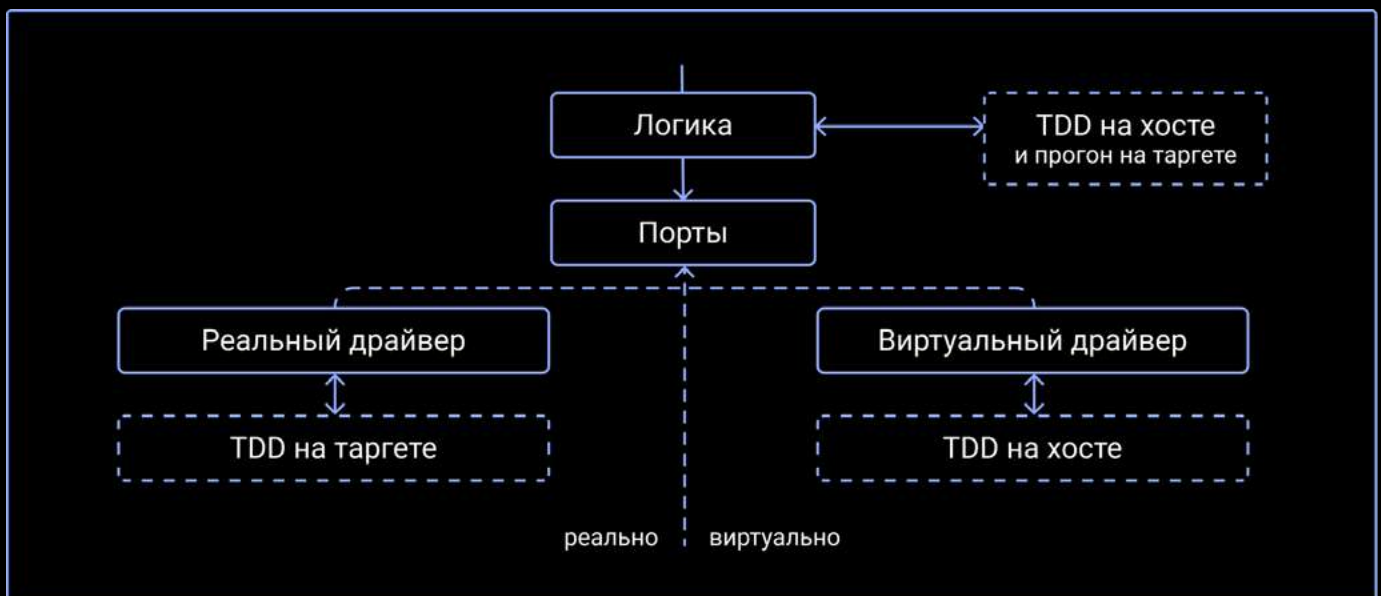
Тестируются на железе, поэтому требуют сборки и прошивки устройства. Работать по TDD на этом уровне остаётся также сложно.

Логика с верхнего уровня не должна тянуть заголовочный файл для общения с платформой — иначе магия не сработает: вы снова связали весь код с железом, тестирование на устройстве неизбежно. Подмена реализации делается через разделение интерфейса и реализации: один интерфейс, две реализации, нужная выбирается в зависимости от цели сборки. Никаких `#ifdef` в логике.

Доля кода на разных уровнях зависит от домена: устройство, которое дёргает реле по внешнему сигналу, почти не содержит логики, поэтому Dual Targeting не сыграет большой роли. А вот в устройстве с расчётами, обновлением данных по сети, обработкой состояний и сложными алгоритмами доля логики может быть очень большой. Значит, большая часть разработки может проходить через быстрый TDD-цикл на хосте.

Тут тесты запускаем у себя локально при каждом TDD-цикле, а собираем код под таргетную платформу и проводим тесты на железе при pull-request в мастер.

Оговорка: гонки с прерываниями, реальные дедлоки и глубину стека тесты на хосте не увидят. Dual Targeting убирает из TDD-цикла лишнее ожидание, а не необходимость проверки на железе.



Бонус: санитайзеры

ASan и TSan на микроконтроллер не затащить, а на хосте они включаются одним флагом. Но помним: проверяют они хостовую сборку, не таргетную.

TDD — это десятки циклов в час. Без Dual Targeting 40 итераций по 10 минут — почти полный рабочий день. На хосте те же 40 могут уложиться в час. Поэтому в embedded вопрос не обязательно звучит как «TDD или железо?». Можно сделать так: быстрый TDD-цикл — на хосте, финальная проверка — на железе.

Где в мире TDD живут тестировщики?

Тактическая забота о качестве и тестировании ложится на плечи каждого разработчика команды. Стратегическая — на плечи QA-инженера.

В таком варианте QA-инженер не просто запускает ручные тесты или пишет автотесты на уже созданный код. Он больше напоминает играющего тренера, который прописывает тестовую стратегию с командой, следит за её исполнением и обучает разработчиков, какие тесты на каком уровне надо писать.

QA-инженер — это коуч, который продумывает, какая система тестирования будет в проекте, и помогает справиться, когда возникает нетривиальная ситуация, где нужно понять, как в этом случае нужно расставить тестовые проверки.

Часто тестовая стратегия на бумаге напоминает идеальную луковицу в разрезе. Все круги идеально расположены и не цепляются друг за друга. Такую луковицу мы видим в рекламе самого вкусного бургера или на конкурсе овощей. В реальной жизни всё бывает не так просто. Иногда луковица выглядит по-другому:



ЛУКОВИЦА В РЕКЛАМЕ



ЛУКОВИЦА ДОМА

Хорошо, когда ваш QA придумал стратегию, которая покрывает большую часть потенциальных вопросов, и в сложной ситуации каждый разработчик идет сверяться с тестовой стратегией и сразу видит ответ на свой вопрос: да, твои тесты на CRUD должны выглядеть так. В идеальной ситуации к этому выбору тестов даже есть объяснение и другие варианты, которые рассматривались, но были отвергнуты.

Но в ситуации с реальной луковицей QA должен помочь разобраться, как сделать хорошо, и как вписать это «хорошо», но «по-другому» в общий рисунок из тестов.

Придумывать, как стандартизировать тесты, как поступать в нетривиальных случаях, предугадывать наперед, какие проблемы могут возникнуть дальше, проверять написанные разработчиками тесты и быть опорой команды и владельца продукта. Освещать путь и делать его бетонированным мостом, а не расшатывающимся верёвочным мостиком. И регулярно проверять, что этот мост в порядке, а не трещит по швам и требует реновации.

В общем, задача QA — системно следить за качеством ПО.

Эпоха ИИ — требования к качеству как никогда высоки

ИИ стремительно ворвался в нашу жизнь, ускорив написание кода. И именно в этот момент самое важное — это проверка его качества.

TDD здесь оказывается в особенно удобной позиции. Тест, написанный до кода, — это по своей сути формулировка задачи:

*«ВОТ ВХОД,
ВОТ ОЖИДАЕМЫЙ ВЫХОД,
ВОТ ГРАНИЧНЫЕ СЛУЧАИ»*

Цикл TDD в работе с ИИ выглядит так: инженер пишет тест и фиксирует поведение, модель генерирует реализацию, тест проверяет результат. Если тест не проходит — модель видит конкретное расхождение между ожиданием и реальностью. При этом весь накопленный капитал из автотестов гарантирует, что другие части системы не были поломаны.

Это ли не ренессанс Test-Driven Development?

Появление ИИ послужило катализатором улучшения структуры и архитектуры: сейчас как никогда важны спецификации, контракты, user story, итеративная разработка и хорошо написанные автотесты, которые смогут гарантировать качество создаваемого продукта. То, что годами считалось накладными расходами, теперь становится способами сделать ИИ еще более полезным и эффективным.

С приходом ИИ появился новый аргумент в пользу TDD: разработка через тестирование — это способ получить от ИИ предсказуемый код вместо правдоподобного. Ведь быстрая кодогенерация — это в том числе и быстрое создание техдолга. А технический долг — это, как известно, проблема бизнеса.

Команда, у которой есть тестовая стратегия и привычка работать по TDD, использует ИИ с большей отдачей, чем команда, которая работает по-другому.



ПРО ТЕХНИЧЕСКИЙ
ДОЛГ

Микрофон в зал: что думает команда



Александр

Руководитель
компании, внедрял
TDD в процессы



Антон

Тимлид embedded-
команды, работал
по TDD с устройствами



Анастасия

Fullstack- разработчик,
3 года TDD в вебе



Снежана

ML/backend-инженер,
сейчас переходит
от тестов после
кода к TDD

Во сколько вам обошёлся переход на TDD?

Александр *Руководитель компании*

Во сколько слезинок? К переходу можно долго идти, постепенно добавляя тестирование в разработку. С одной командой на 5 инженеров это заняло 3 года. В какой-то момент команда уже сама сказала: «Будем работать по TDD». Не пришлось больше убеждать, предлагать провести эксперимент. TDD сложно внедрять сверху вниз, но можно. Говорите: «Теперь мы работаем так». Но если этому не помогать, то ничего не получится. Изменения внедрять и так сложно, потому что всегда есть сопротивление. В общем, переход обошёлся в 5 слезинок, 3 года и 1 большую удачу на коллег, с которыми меня свела судьба. В деньги переведите сами.

Антон *Тимлид embedded-команды*

X часов командного времени на выработку привычки и перестройку подхода к написанию кода, плюс 100500 дополнительных чашек кофе и 10мл слёз тимлида.

Анастасия *Fullstack- разработчик*

Моими «валютами» стали время и нервы, а также долгая борьба с нежеланием писать тесты раньше кода, вместо того, чтобы поскорее реализовать функционал (при всём понимании, насколько это упростит и обезопасит разработку), и постоянное непонимание того, как написать тест, пока окончательно не сложится картина самой реализации.

Снежана *ML/backend-инженер*

Для меня процесс перехода ещё далёк от завершения, но по ощущениям с недавнего проекта — в валюте времени выхожу в ноль. То есть на сколько больше времени ушло в начале на формулировку тестовых сценариев — на столько же меньше ушло на реализацию, потому что уже было понятно, какой функционал нужно охватить.

Какой аргумент «за» TDD все повторяют, а он не работает?



Александр *Руководитель компании*

TDD даёт 100% покрытия кода тестами. Я бы поставил под сомнение, это миф. Не в этом цель. Так может получиться, но на реальных проектах, когда вы применяете TDD, а не превращаете его в карго-культ, может быть покрытие меньше. Потому что вы можете решить что-то вообще не тестировать. TDD же существует не в изоляции, а в связке с тестовой стратегией. И если она не подразумевает полное покрытие, то так и не получится.

Антон *Тимлид embedded-команды*

Экономия времени. Хотя кажется в дальней перспективе всё-таки наблюдается экономия, но на первый взгляд в начале разработки это не ощущается. Команда тоже часто настроена скептически, топит за решение «здесь и сейчас».



Анастасия *Fullstack-разработчик*

Зелёные тесты создают ощущение, что все случаи учтены, хотя на деле это ощущение полноты, а не сама полнота. Список кейсов настолько хорош, насколько хорошо его составил человек, и TDD ничего не добавляет к этому списку сам по себе.



Снежана *ML/backend-инженер*

Наверное, что TDD заставляет думать наперёд и сокращает тем самым число багов впоследствии. Он, может, и работает, но очень важно делать оговорку, что не для всех проектов. Для меня этот аргумент зачастую не работал, потому что многие задачи имели исследовательскую природу без чёткой спецификации ожидаемого результата. Не получается продумать наперёд тесты, когда можно лишь обозначить примерные ориентиры R&D-задач.

Что в TDD раздражает лично вас?



Александр *Руководитель компании*

Что если я руководствуюсь не каноническим TDD, а моей интерпретацией и прагматичным подходом, и не пытаюсь назвать это как-то по-другому, например, ATDD, то тогда бывает, что в профессиональном сообществе мне отказывают в том, что я работаю по TDD, или мне приходится оправдываться за свой подход.

Антон *Тимлид embedded-команды*

TDD врубает тормоза и не даёт сразу писать код. Представьте, что в вашей голове уже созрело гениальное решение, всего-то осталось создать новую функцию и добавить туда пяток строк кода. Но нет, оказывается, что тест задаёт несколько неприятных вопросов, о которых мы почему-то не подумали сразу...



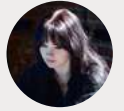


Анастасия *Fullstack-разработчик*

Замедление разработки на старте, при изменении кода нужно всегда думать и о тестах.

Снежана *ML/backend-инженер*

Когда ты думаешь, что уже всё продумал, — это всё равно не так... Вы можете утвердить состав тестов, смерзиться, а потом вылезает новая правка, для которой нужно не забыть дополнить тесты. И приходится усиленно держать себя в руках, чтобы не перепрыгнуть этап с тестами и сразу не «учитывать» эту правку в реализации. Так что одно дело — разработка по TDD в начале жизни проекта, а другое, более раздражающее — соблюдение TDD-цикла в дальнейшем, особенно для небольших фич.



Где TDD вам не помог?



Александр *Руководитель компании*

TDD сам по себе не помог наладить дисциплину команды. TDD сам по себе не делает команду глубокими экспертами в стеке, тестировании. Но TDD может выступить катализатором того, чего не хватает команде: ответственности, вовлеченности, заинтересованности в личном и командном росте инженерной культуры, дисциплины и технической экспертизы.

Антон *Тимлид embedded-команды*

Hardware-зависимый код. Иногда железо решает, что оно «умнее» кода.



Анастасия *Fullstack-разработчик*

Баг, который почти нельзя воспроизвести, потому что он связан на внешние процессы и окружение. Написать тест, воспроизводящий такое до фикса, почти нереально, и правило «сначала красный тест» становится абсурдным.

Снежана *ML/backend-инженер*

Не то что «не помог», скорее, было не понятно, как его применить. Так бывает на R&D-проектах, где есть концепция желаемого результата, но путь к нему неизвестен. Что конкретно тестировать, как это будет работать, что мы вообще разрабатываем? На этапах проверки гипотез неразумно писать тест на функцию, которая завтра станет бесполезной. Ещё TDD не удалось применить на телеграм-боте: мы не знали, как тестировать ботов, а дедлайны подгорали. TDD и так требует больше времени в начале проекта, а если методы тестирования не знакомы — тем более.



В какой момент вы поверили в TDD?

Александр *Руководитель компании*

Я поверил? Я не такой наивный. Я попробовал и посмотрел, как работает. Скачок из «тестов после кода» в «тесты до кода» произошел, когда я послушал доклад коллеги про ценность TDD для качества продукта и инженерной культуры. Он сказал: «У нас как у инженеров нет оправдания, почему мы не работаем по TDD». Я подумал: «Ну да, нам ничего не мешает. Осталось начать так работать принципиально».

А поверил уже после внедрения в команды компании, когда за 3 месяца разработки тайм-трекера не увидел ни одного бага на проде. Всё было стабильно даже без QA.



Антон *Тимлид embedded-команды*

Когда пришли очередные правки от заказчика и потребовалось менять логику некоторых модулей. Поменяли реализацию, прогнали тесты — и сразу увидели, что сломалось, а что продолжило работать как раньше. Двигаемся уверенно, не большими шагами.



Анастасия *Fullstack-разработчик*

Поверила в идею на старте. Верю больше каждый раз, особенно при рефакторингах со всеми зелёными тестами. Несмотря на трудности поначалу, на дистанции TDD даёт уверенность в работоспособности разрабатываемой системы.



Снежана *ML/backend-инженер*

Первой на ум приходит ситуация, когда нам понадобилось использовать в разработке нестандартное решение, у которого был практически единственный недостаток: ошибку можно будет обнаружить только при вызове конкретного эндпоинта. Мы смогли пойти на это решение, потому что были уверены: все эндпоинты вызываются при запуске E2E-тестов, то есть ошибка в любом случае не останется незамеченной и не «утечёт» на прод.



Как начать работать по TDD?

Мы знаем, как нелегко переводить команду на Test-Driven Development после многих лет работы с другими привычками. Но кажется, что сейчас самое время для того, чтобы пробовать.

Начать можно с малого: сначала писать тесты на найденный баг. Нашли баг, зафиксировали его автотестом и только после этого починили. Через месяц такой работы у команды есть набор регрессионных тестов и привычка начинать работу с теста. Это будет легким входом.

И пусть вам помогает Мартин Фаулер и его цитата:

«Никакой баг не считается исправленным без автоматического регрессионного теста.

В культуре, где есть тесты, когда баг находят, естественная реакция — написать тест, который его обнажает, а потом починить код».

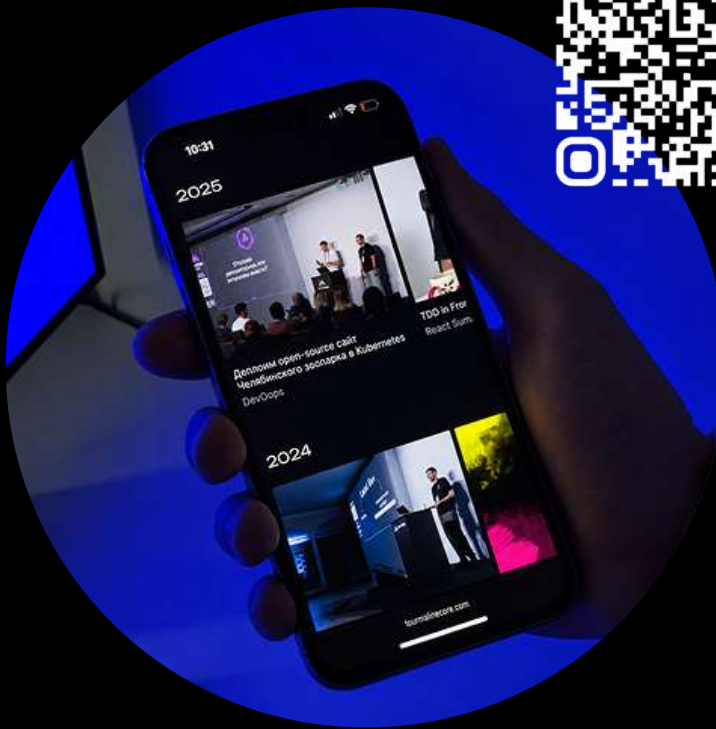
И ещё Кент Бек в помощь:

«TDD убирает страх из разработки. А тесты дают разработчику смелость менять, удалять, добавлять».



Можно также написать нам и поболтать про TDD: будем рады поделиться опытом.

КАРТА НАШИХ ОТКРЫТЫХ МАТЕРИАЛОВ ПО ТЕМЕ



В журнале конспект, а за QR-кодом — все материалы по теме, накопленные за 7 лет нашего пути к TDD. Доклады с конференций, воркшопы, мини-курс по .NET-бэкенду по TDD и открытые репозитории проектов, о которых мы рассказывали.



ПОДЕЛИТЬСЯ МЫСЛЯМИ
ИЛИ ОСТАВИТЬ ОТЗЫВ



УЗНАТЬ БОЛЬШЕ
О КОМПАНИИ